Duoc UC – Sede Maipú

Capstone 2026 – Asignatura APT122

Evento Cultural

Documentación Técnica del Proyecto

Plataforma chilena de eventos culturales

Integrante	Rol principal
Roberto Riffo	Líder de proyecto / Frontend / UX
José Miguel Moncada	Base de datos / QA
David Herrera	Backend e integraciones

Roles según el design brief aprobado en Fase 1 (*design_brief_capstone.docx*).

Stack del sistema: Python (scraper con conectores, NVIDIA NIM y checkpoints SQLite), Supabase/PostgreSQL con Row Level Security, Edge Functions (registro, cuenta, publicación, moderación y recuperación), Angular 22 + Ionic 9 (web y móvil, catálogo real y autenticación directa vía `@supabase/supabase-js`), Leaflet + OpenStreetMap (mapa interactivo con pines georreferenciados) y GitHub Actions para automatización.

Versión 1.3

31 de octubre de 2026

Revisión de cierre (Fase 3): agrega estándares de desarrollo, estructura de evidencias y referencias.
Revisión contrastada con el código local actual; resultados históricos separados
de las verificaciones de cierre pendientes. Ver Respuesta_Rubrica_Fase2.md.
Las omisiones se declaran como **pendientes del equipo** dentro de este mismo documento.

Índice

1. Introducción y alcance **2**
- 1.1. Estructura del sistema . . . 2
- 1.2. Respuesta a la evaluación anterior . . . 2

2. Arquitectura general del sistema **2**
- 2.1. Alcance y fuente de verdad . . . 2
- 2.2. Contratos y límites de confianza . . . 3

3. Arquitectura del scraper **4**

4. Arquitectura del frontend **5**
- 4.1. Stack y organización . . . 5
- 4.2. Descubrimiento y personalización . . . 6
- 4.3. Cuenta y perfil . . . 6
- 4.4. Administración y publicación . . . 6
- 4.5. Despliegue y pendientes . . . 6

5. Modelo de datos (Supabase / PostgreSQL) **6**
- 5.1. Fuente del modelo . . . 6
- 5.2. Tablas y finalidad . . . 8
- 5.3. Seguridad (Row Level Security) y ciclo de carga . . . 9
- 5.4. Limitaciones declaradas . . . 9

6. Diagramas UML **9**
- 6.1. Casos de uso . . . 10
- 6.2. Secuencia de la funcionalidad principal . . . 11
- 6.3. Componentes y clases . . . 11
- 6.4. Pendientes de UML . . . 12

7. Requisitos no funcionales **12**
- 7.1. Seguridad y limitaciones . . . 13

8. Pruebas y evidencia **13**
- 8.1. Resultados históricos conservados . . . 13
- 8.2. Plan incremental de cierre . . . 14

9. Despliegue sin Docker **14**

10. Innovación y alcance del MVP **14**
- 10.1. Sostenibilidad: propuesta futura . . . 14

11. Estándares de desarrollo y evidencia del avance **14**
- 11.1. Estándares y prácticas de la industria aplicados . . . 14
- 11.2. Estructura de evidencias, nomenclatura y control de versiones . . . 14
- 11.3. Roles planificados y trazabilidad individual pendiente . . . 15

12. Trazabilidad documental **15**

Referencias **15**

1. Introducción y alcance

Este documento consolida el estado técnico de Evento Cultural al 31-10-2026 a partir del código local actual, las funciones Edge y configuración de la aplicación. Conserva como línea base los requerimientos de Fase 1, el modelo de datos y la evidencia del scraper y QA de Fase 2. No certifica el esquema remoto ni reemplaza una ejecución de pruebas.

Convención de honestidad documental: este documento distingue explícitamente entre lo *implementado* (respaldado por código), lo *parcial* (respaldado solo en parte) y lo *pendiente* (información humana o trabajo futuro). Nada se afirma como terminado sin respaldo; los puntos abiertos se marcan con **Pendiente de completar por el equipo.**

1.1. Estructura del sistema

El sistema se compone de tres bloques independientes que se comunican a través de la base de datos: el **scraper** (recolección y enriquecimiento de datos), la **base de datos** (catálogo canónico en Supabase/PostgreSQL) y el **frontend** (aplicación web y móvil), con **GitHub Actions** como automatización de la ejecución del scraper (ver Figura 1 en la Sección 2).

1.2. Respuesta a la evaluación anterior

La planilla de Fase 2 registra 74/100 y nota 5,1. Los descuentos están en los indicadores 3 (15/25), 4 (3/5), 5 (3/5), 6 (12/20) y 8 (6/10). No contiene observaciones narrativas específicas. Se corrigen precisión técnica, estado del producto, fuentes y trazabilidad; la entrega en Git y el dominio oral del equipo requieren evidencia adicional. El registro `Respuesta_Rubrica_Fase2.md` vincula cada indicador con acciones sin prometer una nueva calificación. Código implementado no equivale a prueba aprobada.

2. Arquitectura general del sistema

2.1. Alcance y fuente de verdad

Esta revisión toma como fuente principal el código local de `EventoCultural_Integrado/EventoCultural_Integrado`. La rama de referencia es `https://github.com/sunsitox/Proyecto_Capston_Evento_Cultural/tree/deploy/vercel-preview`. No se certifica igualdad entre la copia local y esa rama ni que todas las funciones locales estén desplegadas. Se distingue implementación, configuración remota y verificación de extremo a extremo.

- **Recolección:** scraper Python independiente, con conectores, saneamiento, NVIDIA NIM selectivo y checkpoints SQLite. Su evidencia operativa conservada corresponde al ciclo del 11-10-2026; no constituye una nueva ejecución del scraper.
- **Persistencia:** Supabase/PostgreSQL, catálogo y perfiles con políticas RLS; Supabase Auth para sesiones y Storage para avatares.
- **Cliente:** SPA Angular 22 e Ionic 9, con servicios TypeScript y signals. Consulta catálogo y autenticación mediante `@supabase/supabase-js`. El código Python de `backend/` es legado y no es una dependencia del flujo web actual.
- **Servidor:** Supabase Edge Functions para operaciones privilegiadas: `registrar-cuenta`, `admin-account`, `publicar-evento`, `admin-eventos`, `reset-password-request`, `reset-password-confirm` y `recomendaciones-semanal`. La clave de servicio pertenece exclusivamente al servidor.
- **Entrega:** Vercel para la SPA, GitHub Actions para el scraper y Capacitor para Android. No se utiliza Docker en la estrategia adoptada.

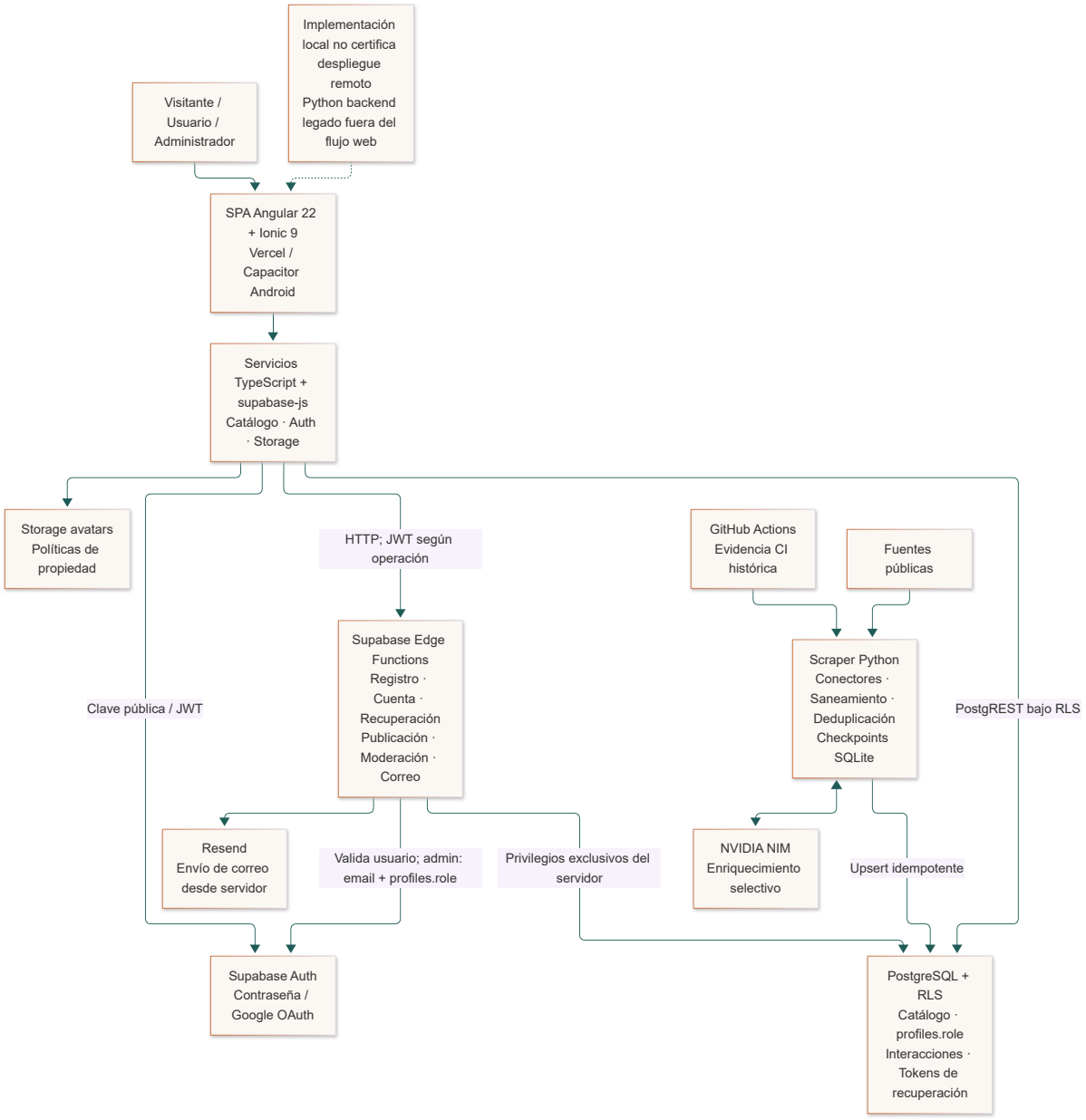


Figura 1: Arquitectura actual; fuente editable: Diagramas/01_arquitectura_general.mmd.

2.2. Contratos y límites de confianza

Origen y destino	Contrato	Regla de acceso
Cliente a Supabase	Catálogo mediante PostgREST; sesión mediante Auth	Clave pública y JWT del usuario; las políticas RLS gobiernan los datos.
Registro a función Edge	Correo, contraseña y nombre	Crea cuenta confirmada mediante Admin API; luego el cliente inicia sesión. No comprueba la propiedad del buzón.
Usuario a publicar-evento	Datos validados del evento y JWT	Identidad obtenida con getUser; crea evento draft y no acepta status/source_id del cliente.

Origen y destino	Contrato	Regla de acceso
Administrador a admin-eventos	Listado y acciones de moderación	Valida JWT, proveedor email y profiles.role=admin antes de utilizar privilegios de servicio.
Recuperación a funciones Edge	Solicitud por correo y confirmación con token	JWT firmado y registro password_reset_tokens con caducidad y marca de uso; requiere migración y secretos.
Servidor a correo	API de Resend	Clave privada en el entorno de funciones. Envío implementado no equivale a recepción demostrada.

3. Arquitectura del scraper

El pipeline carga fuentes declarativas, selecciona conectores por Strategy/Factory, sanea y deduplica eventos, normaliza ubicaciones, exporta JSON/CSV y sincroniza Supabase por upsert. Los checkpoints SQLite permiten reanudar trabajo y reutilizar resultados de IA de eventos sin cambios. El límite configurado de NVIDIA es 40 RPM; no se presenta como una garantía contractual del proveedor.

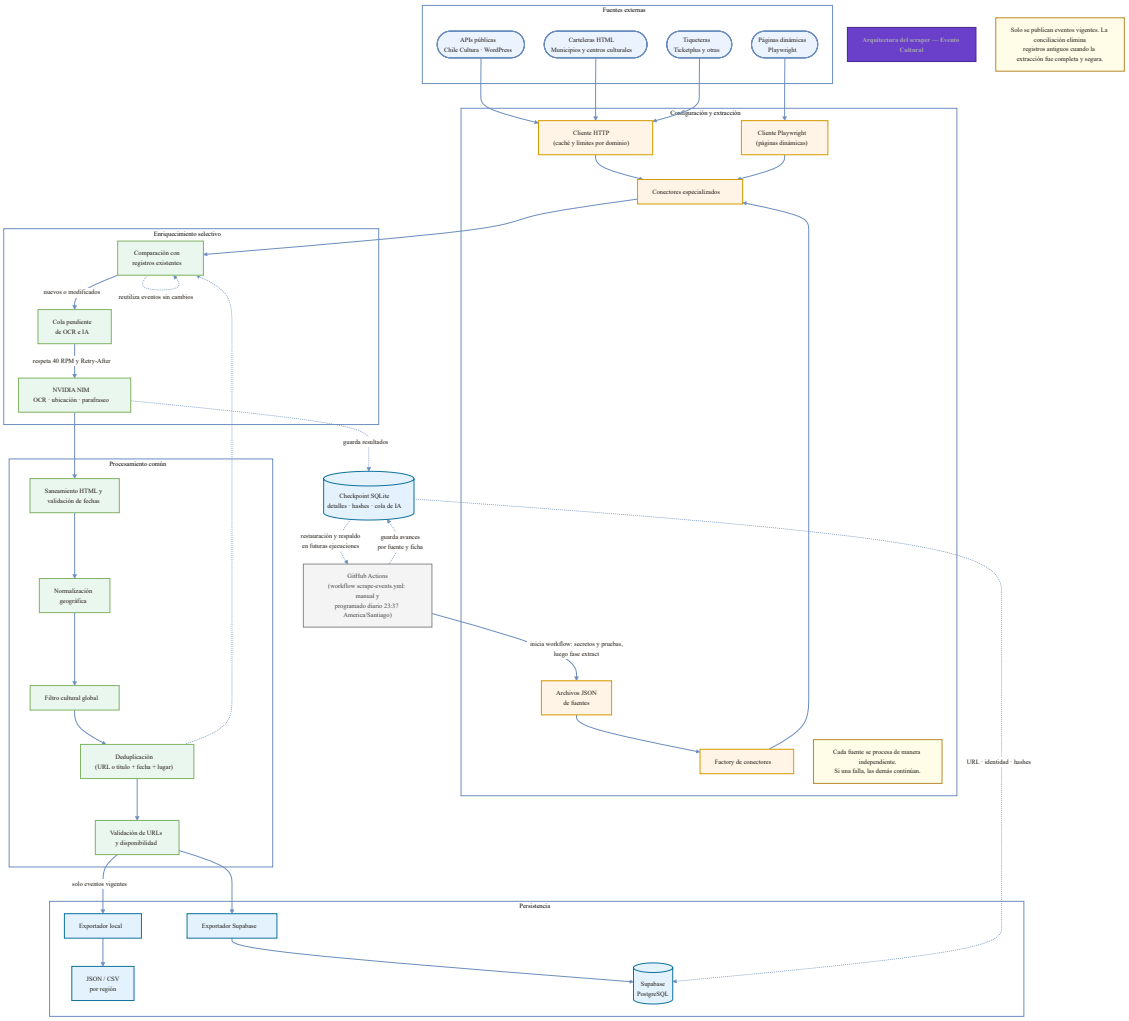


Figura 2: Pipeline del scraper conservado de la línea base; verificar su inventario vigente antes del cierre.

El inventario de 21 fuentes permanentes y una estacional y las ejecuciones de CI corresponden a evidencia histórica. No se infiere que todas sigan disponibles hoy. Los conectores aíslan fallos por fuente; la conciliación y el borrado de vencidos dependen de la configuración y de una extracción completa. GitHub Actions no garantiza puntualidad: el desfase del run 25 sigue registrado como DEF-CI-001.

4. Arquitectura del frontend

4.1. Stack y organización

Según `package.json`: Angular 22.0.1, Ionic 9, TypeScript 6, Capacitor 8, supabase-js 2, Leaflet 1.9.4, RxJS 7.8 y Vitest 4 con jsdom. `core/` concentra modelos y servicios; `pages/` las pantallas; `features/home/` los layouts; y `layout/` la navegación adaptativa. Las versiones resueltas deben reproducirse con `package-lock.json`.

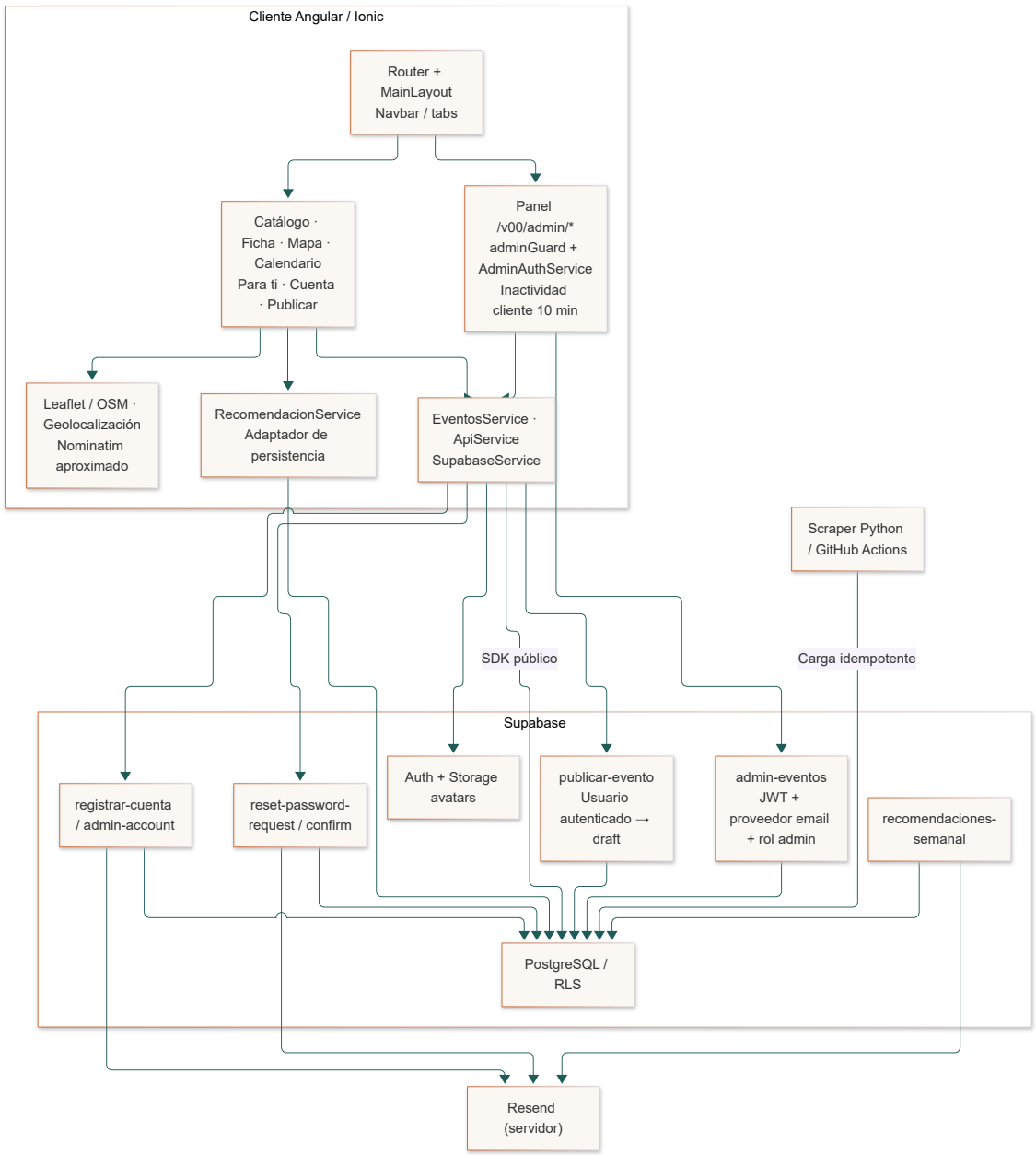


Figura 3: Componentes actuales y separación cliente/servidor.

4.2. Descubrimiento y personalización

El catálogo publicado alimenta búsqueda, filtros combinados, detalle, calendario y mapa Leaflet/OpenStreetMap. Nominatim aporta geocodificación de respaldo con cola, caché y límites; los resultados aproximados no sustituyen coordenadas verificadas. La geolocalización usa Capacitor en Android y alternativa web. Las recomendaciones se calculan por señales de interés, no por un modelo generativo: `RecomendacionService` calcula afinidad y razones; el adaptador de persistencia almacena interacciones en `user_event_interactions` y preferencias en metadata. Los invitados mantienen estado en memoria. El respaldo de eventos de demostración no debe confundirse con una consulta remota exitosa.

4.3. Cuenta y perfil

El registro principal llama `registrar-cuenta`, que crea el usuario con `email_confirm=true`; el cliente ejecuta `signInWithPassword` sin exigir verificación del buzón. La función intenta enviar un correo de registro exitoso por Resend. Existe respaldo legacy con `signUp`: si la función no está disponible, puede reaparecer la confirmación de Supabase. Por tanto, la experiencia solicitada requiere desplegar y configurar la función, no solo cambiar el formulario.

El perfil permite editar nombre, contraseña e imagen. Storage utiliza el bucket `avatars`; el metadata `avatar_url_custom` tiene prioridad sobre la imagen OAuth, y esta se utiliza si no hay imagen propia. La identidad reactiva alimenta perfil y navegación. Google OAuth tiene botón y callback implementados; su configuración remota y la recepción de correos no se certifican por inspección de código. La recuperación actual usa las funciones `reset-password-request/confirm` y `/restablecer-contrasena?token=...`, no únicamente el flujo antiguo de GoTrue.

4.4. Administración y publicación

Las rutas `/v00/admin/*` requieren `adminGuard`; no enlazarlas públicamente no constituye una medida de autorización. `AdminAuthService` consulta `profiles.role` y solo admite `admin`, con proveedor email: visitor, editor y Google son rechazados. El rol editable de metadata no es autoridad administrativa. La función `admin-eventos` repite las comprobaciones en el servidor. El cliente cierra la sesión tras 10 minutos sin interacción, comprobados cada 15 segundos, y redirige al ingreso con aviso. La última actividad se reinicia al recargar; no es un timeout absoluto persistido ni una revocación impuesta por el servidor.

La publicación autenticada crea un borrador mediante `publicar-evento`; `admin-eventos` implementa listado administrativo, publicación y supresión. El dashboard calcula métricas del catálogo cargado en el cliente: no equivale a analítica comercial ni a estadísticas certificadas de uso. La ocultación de fuentes en el navegador no equivale a detener el scraper ni a persistir una decisión global.

4.5. Despliegue y pendientes

`angular.json` genera `www`; `vercel.json` ya incluye la reescritura SPA a `/index.html`. Existe proyecto Android y APK debug local; esto no demuestra instalación, GPS u OAuth en dispositivo físico ni publicación en tienda. Quedan por registrar: regresión del ciclo QA histórico, pruebas remotas de autorización, correo y recuperación, aislamiento entre usuarios, pruebas nativas y mediciones de rendimiento/accesibilidad. Ninguno de esos resultados se deduce de una compilación.

5. Modelo de datos (Supabase / PostgreSQL)

5.1. Fuente del modelo

Corte de Fase 3 (31-10-2026): el ER representa la línea base del catálogo, no un volcado del esquema remoto actual. El código incorpora perfiles para autorización, interacciones para

recomendaciones, Storage de avatares y la migración local `20261024000100_password_reset_tokens.sql`. La tabla de tokens de recuperación almacena identificador, usuario, correo, vencimiento y marca de uso; su despliegue se debe verificar antes de probar recuperación. Auth referencia realmente `auth.users`; el nombre `auth_users` del dibujo es una representación conceptual.

El modelo se documenta a partir de la documentación de base de datos incluida en el entregable del scraper de Fase 2, el DBML editable, el diccionario de tablas (DOCX, versión documentada al 03-10-2026) y el diagrama ER de Fase 1 (`DB_evento_cultural.pdf`). Las migraciones SQL no se incluyeron en la copia local entregada, pero están disponibles en el repositorio oficial del proyecto (https://github.com/sunsitox/sunsitox-Scraper_de_eventos_culturales).

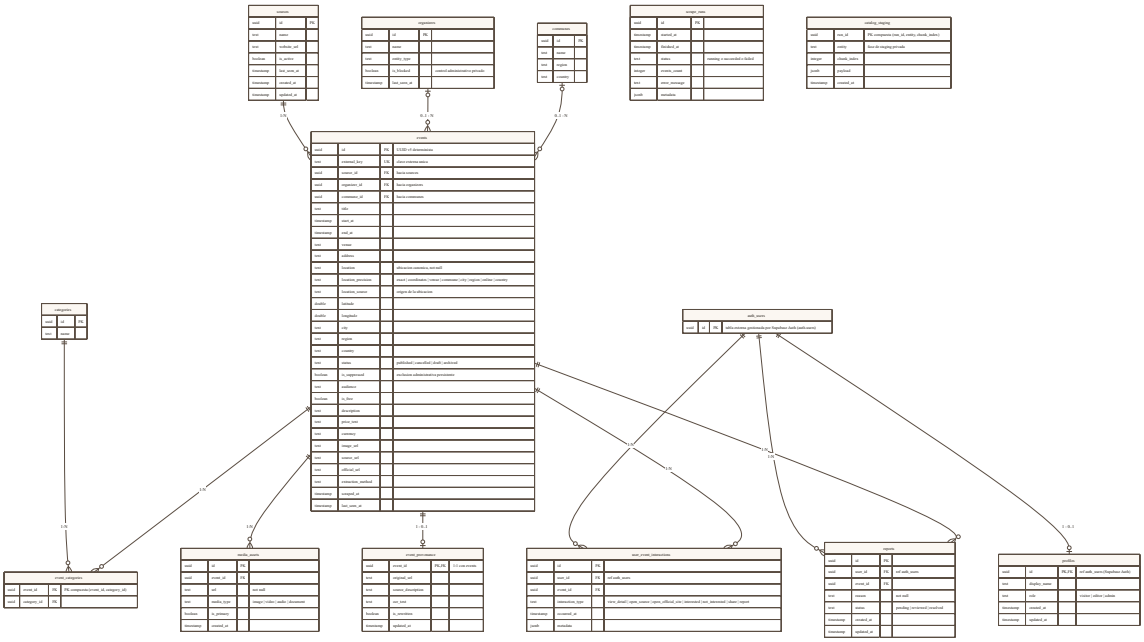


Figura 4: Modelo entidad–relación (fuente editable: `Diagramas/06_modelo_er.mmd`, Mermaid).

5.2. Tablas y finalidad

Los identificadores de eventos, organizadores, comunas y categorías son **UUID v5 deterministas** derivados de claves externas: ejecutar dos veces el mismo scraping actualiza las filas (upsert) sin crear duplicados. El campo `categories` se normaliza en `categories` y `event_categories`, conservando la relación muchos-a-muchos.

Grupo	Tabla	Finalidad
Ingesta y catálogo	`sources`	Agenda, municipalidad, ticketera o sitio de origen
	`organizers`	Institución responsable del evento
	`communes`	Ubicación administrativa normalizada
	`categories`	Etiquetas temáticas, de formato o audiencia
	`events`	Registro canónico del evento consolidado (37 columnas)
	`event_categories`	Relación N:M entre eventos y categorías
	`media_assets`	Imágenes y futuros multimedia
	`scrape_runs`	Bitácora de ejecuciones locales y de GitHub Actions
	`event_provenance`	Evidencia privada: URL original, descripción de la fuente, texto OCR y marca de reescritura
Aplicación	`profiles`, `user_event_interactions`, `reports`	Perfiles e interacciones usados por la aplicación; reports no se declara como flujo completo; `profiles` referencia a `auth_users`, la tabla externa de Supabase Auth
Staging	`catalog_staging`	Área privada de carga por lotes (payload JSON) antes de publicar el catálogo

Cuadro 2: Tablas del modelo de datos según el diccionario y el DBML del proyecto.

5.3. Seguridad (Row Level Security) y ciclo de carga

La migración de RLS habilita Row Level Security en todas las tablas; `scrape_runs` no tiene lectura pública; el usuario no puede cambiar su propio rol ni el estado de sus reportes desde el frontend; la vista `public_event_catalog` hereda las reglas RLS y no expone eventos suprimidos ni organizadores bloqueados.

En el ciclo de carga, la sincronización identifica coincidencias por URL o clave compuesta, carga por lotes (`SUPABASE_BATCH_SIZE`) y registra cada ejecución en `scrape_runs` como `running`, `succeeded` o `failed`. La repetición idempotente es el mecanismo de recuperación ante cargas parciales.

5.4. Limitaciones declaradas

Sin marca explícita de eventos de día completo; `price_text` no permite comparación numérica; categorías sin jerarquía almacenada; registros de Chile Cultura que pueden ser directorios y no eventos fechados; etiqueta inválida conocida (`Calendar_Month Inscribirse`); borrado físico de vencidos solo con `SUPABASE_DELETE_EXPIRED=true`.

6. Diagramas UML

Los cuatro diagramas UML exigidos por el anexo metodológico se escriben como código fuente Mermaid (`.mmd`) y se renderizan a PDF con `mermaid-cli`, en la carpeta `Diagramas/` de esta misma sección. Los casos de uso se derivan de la matriz MoSCoW de Fase 1 y de las páginas realmente implementadas en el frontend. Todos los diagramas (01–07) comparten un estándar visual único (paleta institucional, tipografía Arial, flechas reforzadas), declarado dentro de cada archivo `.mmd`, y fueron regenerados el 27-09-2026 al migrar de Draw.io a Mermaid para automatizar el trazado de líneas y facilitar el mantenimiento; el diagrama entidad–relación (06) fue corregido y reexportado

el 03-10-2026 para alinearlo con el diccionario de tablas y el DBML vigentes; los diagramas 01 (arquitectura general), 02 (componentes) y 04 (secuencia) fueron regenerados el 10-10-2026 para reflejar la integración del sistema: la API intermedia (`supabase_backend.py`) entre el frontend y Supabase, la autenticación real con regreso OAuth en `/auth/callback` y el mapa interactivo. En esta revisión los diagramas 01, 02 y 04 sustituyen la API Python histórica por SDK directo y funciones Edge.

6.1. Casos de uso

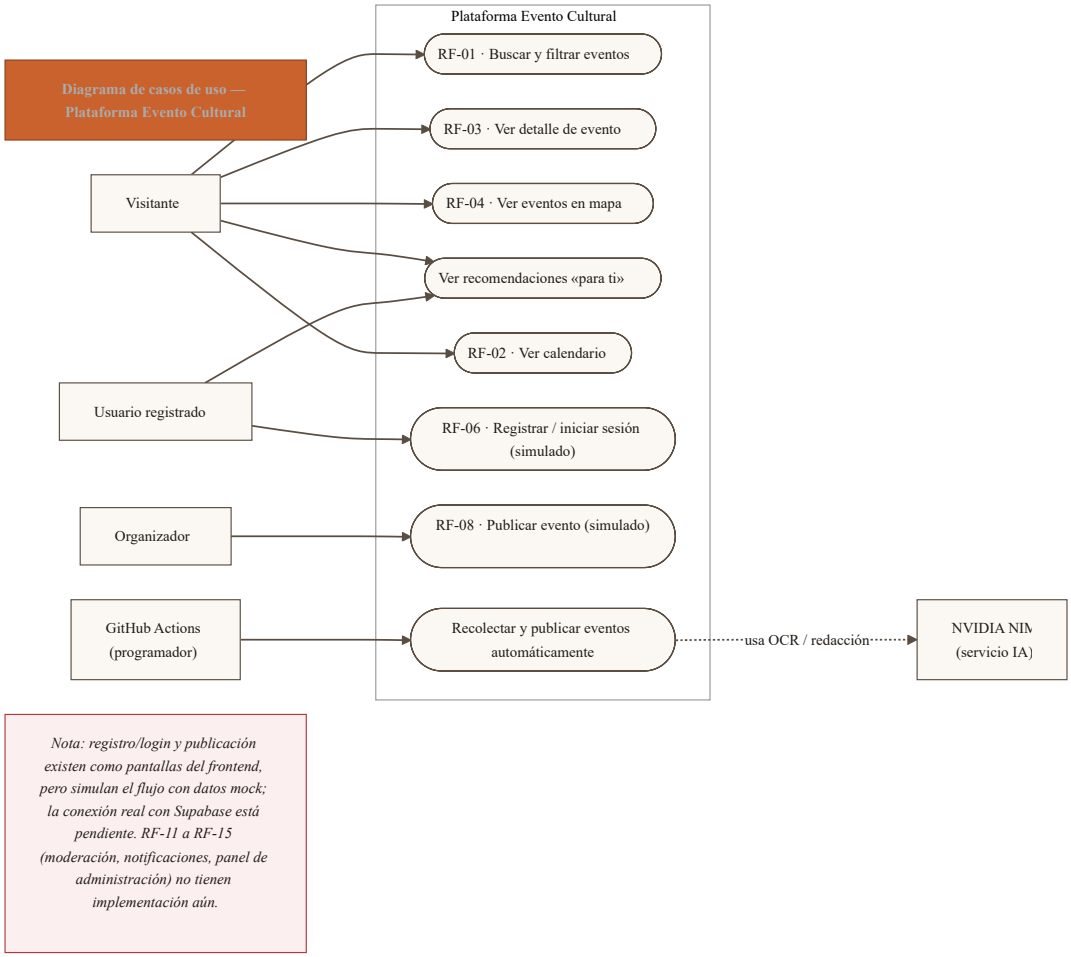


Figura 5: Casos de uso de la plataforma (fuente editable: `Diagramas/03_casos_de_uso.mmd`, Mermaid).

El visitante consulta catálogo, mapa y calendario; la cuenta autenticada gestiona su perfil, recibe recomendaciones y envía borradores mediante publicar-evento. El administrador de proveedor email y rol admin modera mediante admin-eventos. El diagrama de casos de uso conservado es conceptual e histórico, no un inventario exhaustivo de los flujos nuevos. GitHub Actions activa el pipeline y NVIDIA NIM participa en el enriquecimiento.

6.2. Secuencia de la funcionalidad principal

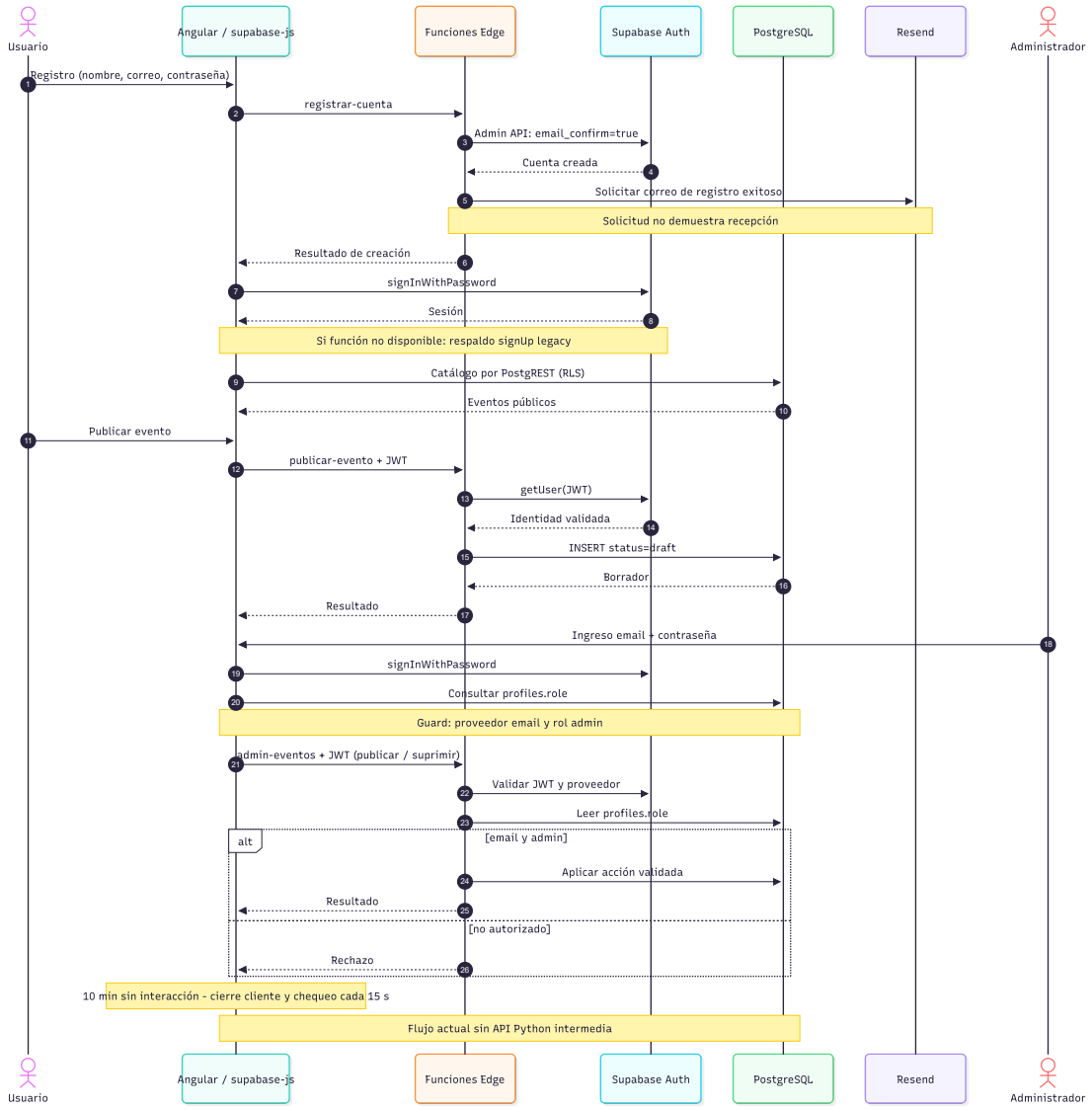


Figura 6: Secuencia de registro, sesión, publicación y moderación (editable: Diagramas/04_secuencia.mmd).

El flujo principal del sistema es la obtención y publicación de eventos: GitHub Actions ejecuta el pipeline; este restaura el checkpoint, consulta las fuentes con caché condicional, sanea y deduplica, decide mediante hash qué eventos pasan por IA (con límite de 40 RPM), reutiliza resultados previos y sincroniza Supabase con upserts idempotentes, registrando la corrida en `scrape_runs`.

6.3. Componentes y clases

El diagrama de componentes (Figura 3) muestra los módulos del frontend y del scraper y su comunicación con la base de datos. En el frontend destacan además el mapa interactivo real (Leaflet sobre tiles de OpenStreetMap, con pines ligados al catálogo, popups y geolocalización opcional del usuario) y la página de regreso OAuth (/auth/callback) integrada en el router. El diagrama de clases (conceptual) refleja los roles reales del núcleo del scraper verificados en el entregable de Fase 2: `EventPipeline`, `ConnectorFactory`, `Connector` (Strategy), `Event`, `EventSanitizer`, `LocationNormalizer`, `NvidiaService`, `SupabaseExporter`, `DatasetExporter`

y `PipelineState`.

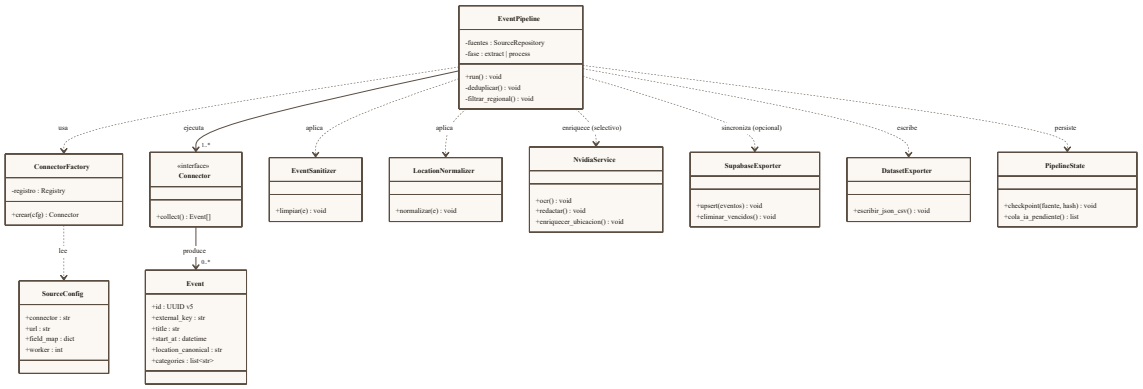


Figura 7: Diagrama de clases conceptual del núcleo del scraper (editable: `Diagramas/05_clases.mmd`, Mermaid).

6.4. Pendientes de UML

- **Actualizado (31-10-2026):** los diagramas 01, 02 y 04 se ajustan a Supabase directo y funciones Edge; la API intermedia Python no pertenece al flujo actual.
- Si el docente exige notación UML estricta: los diagramas de clases (05) y casos de uso (03) son versiones conceptuales en Mermaid; una regeneración formal añadiría atributos y multiplicidades por clase, y relaciones `«include»`/`«extend»` con actores delimitados. La fuente editable (`Diagramas/*.mmd`) es el punto de partida.
- Diagrama de clases del frontend (opcional): complementar el (05) con los servicios Angular (`EventosService`, `ApiService`, `RecomendacionService`, `ViewPreferencesService`). **Pendiente de completar por el equipo:** confirmar la exigencia del docente, definición de responsable y fecha.

7. Requisitos no funcionales

Se conserva la identificación RNF de la matriz MoSCoW. Implementación no equivale a cumplimiento medido: los estados siguientes se basan en código local y evidencia histórica identificada, no en una auditoría actual de producción.

ID	Estado	Fundamento y brecha
RNF-01	Parcial	Interfaz de descubrimiento y cuenta implementada. Revalidar defectos de usabilidad del ciclo 11-10-2026.
RNF-02	Parcial	Navegación adaptativa y proyecto Capacitor con APK debug existente. Faltan pruebas nativas en dispositivo y regresión responsive.
RNF-03	Sin medición actual	El tiempo HTTP histórico no mide renderizado ni Core Web Vitals. Faltan volumen, concurrencia y métricas de interacción.
RNF-04	Parcial	Auth, funciones Edge, RLS y secretos de servidor. Registro principal sin verificación de propiedad del buzón; revisar abuso y exposición de datos. La presencia de políticas no acredita su eficacia remota.

ID	Estado	Fundamento y brecha
RNF-05	Parcial	Guard y servidor exigen profiles.role=admin y proveedor email; visitor/editor/OAuth quedan fuera. Timeout de 10 min en cliente. Faltan pruebas negativas y de revocación.
RNF-06	Parcial	Vercel y evidencia histórica de CI. No se aporta medición de disponibilidad actual ni SLA; el scheduler puede retrasarse.
RNF-07	Diseño implementado	Paginación y upsert en componentes documentados. No se acredita capacidad bajo carga sin medición.
RNF-08	Parcial	Pipeline de saneamiento y deduplicación documentado; revisar muestra actual del catálogo y ubicaciones aproximadas.
RNF-09	Parcial	Servicios, pruebas Vitest, fuentes de diagramas y documentación versionable. Completar ejecución actual y trazabilidad de entrega.
RNF-10	Disponible	Contenido en español y regiones/comunas chilenas; revisión editorial final pendiente.
RNF-11	Parcial	Build web, configuración Vercel y Capacitor presentes; validar reproducción en entorno limpio. Docker no adoptado.
RNF-12	Parcial	Roles ARIA, foco y movimiento reducido en formularios. No se declara conformidad WCAG 2.2 completa sin auditoría y recorrido manual.

Cuadro 3: Estado de los RNF al corte documental.

7.1. Seguridad y limitaciones

La autenticación comprueba identidad; la autorización administrativa se decide con `profiles.role` y se repite en `admin-eventos`. Ocultar una ruta no la protege. La clave pública del navegador no es un secreto y exige políticas RLS correctas; `service_role` no debe incorporarse al frontend, capturas ni Git. El registro sin verificación de correo es una decisión funcional: confirmar mediante Admin API no acredita que la persona controle el buzón. El correo de bienvenida no reemplaza esa comprobación. El cierre por inactividad es cliente y puede reiniciarse con recarga; no se describe como revocación de servidor.

Las notificaciones por correo tienen código en funciones, pero requieren configuración, recepción comprobada y, para recomendaciones periódicas, invocación programada verificable. La moderación de eventos está implementada; los reportes ciudadanos y la administración de organizadores no se dan por completos por existir tablas o rutas. La ocultación local de fuentes no cambia la ejecución del scraper. La estadística del dashboard es un cálculo del catálogo, no analítica comercial.

8. Pruebas y evidencia

8.1. Resultados históricos conservados

El ciclo QA de Vercel del 11-10-2026 registra 13 escenarios: 6 aprobados, 6 con observaciones y 1 fallido, con 13 defectos abiertos. Se conserva en `Evidencias_QA_Frontend_2026-10-11` y en la matriz metodológica. No se cierran defectos sin una nueva ejecución. La evidencia de CI del scraper conserva cinco ejecuciones trazadas y el run 25 funcional con dos artefactos; su inicio fuera de ventana se registró como DEF-CI-001. El inventario histórico describe 14 módulos de pruebas del scraper; no se certifica que ese conteo represente la rama actual.

El anexo previo registra `ng test -watch=false` con 23/23 el 27-10-2026 y builds anteriores. Son antecedentes, no resultados reproducidos en esta revisión documental. Los archivos actuales

incluyen pruebas de recomendaciones, Supabase y acciones de interés. No se infiere cobertura de seguridad a partir de su existencia.

8.2. Plan incremental de cierre

El Plan de pruebas metodológico incorpora casos F3-01 a F3-10 con resultados **pendientes**. Incluyen registro/bienvenida, recuperación, perfil, roles, OAuth, inactividad, moderación, persistencia, Android y regresión. Los comandos previstos son `npm test - -watch=false`, `npm run build` y las pruebas focalizadas de cada integración. Una ejecución debe identificar versión, entorno, responsable y resultado real; la compilación de esta documentación únicamente acredita generación de entregables.

9. Despliegue sin Docker

La estrategia adoptada no utiliza Docker: el scraper se ejecuta mediante GitHub Actions, la SPA en Vercel y Android con Capacitor. Supabase es un servicio administrado. No se agrega un despliegue alternativo ni se declara Docker como implementado. Su adopción futura requeriría una decisión explícita de arquitectura y pruebas propias. El Manual técnico describe la reproducción del build y las dependencias de servidor.

10. Innovación y alcance del MVP

El valor del proyecto es reunir fuentes culturales heterogéneas y conservar procedencia, normalización, deduplicación y vigencia en un catálogo consultable. El uso selectivo de NVIDIA NIM para OCR y enriquecimiento evita repetir trabajo sin cambios; los checkpoints permiten reanudar procesos. Las recomendaciones al ciudadano se basan en afinidad y señales explícitas; no se presentan como inferencia de un LLM.

El prototipo académico depende de servicios externos y cuotas configuradas; no dispone de una infraestructura propia de inferencia. No se afirman ahorros, mejoras de latencia, usuarios activos ni cifras de eventos actuales sin medición fechada. El mapa diferencia coordenadas conocidas y ubicaciones aproximadas; esas aproximaciones no deben presentarse como direcciones exactas.

10.1. Sostenibilidad: propuesta futura

El MVP es gratuito y no implementa pagos, suscripciones, venta de datos personales ni comisiones por entradas. Un eventual contenido patrocinado exigiría validación legal, comercial y ética, identificación visible, separación de recomendaciones orgánicas y compatibilidad con las condiciones del hosting. No se certifican condiciones comerciales vigentes de un proveedor mediante este documento. Cualquier monetización queda fuera del alcance del cierre actual.

11. Estándares de desarrollo y evidencia del avance

Esta sección declara los estándares de la industria bajo los que se desarrolla el proyecto, cómo se organiza la evidencia del avance bajo control de versiones y dónde pueden verificarse los aportes por integrante; responde así a los criterios de evaluación de .evidencias que cumplan estándares de desarrollo", .estructura y nombresz .aporte por integrante".

11.1. Estándares y prácticas de la industria aplicados

11.2. Estructura de evidencias, nomenclatura y control de versiones

La carpeta raíz del proyecto se organiza por fase según la estructura oficial del curso ("Resumen evidencias.xlsx"): `Fase 1/2/3`, cada una con sus `Evidencias Individuales/`, `Evidencias`

`Grupales/` y `Evidencias Proyecto/` según aplique. La nomenclatura es uniforme: nombres descriptivos en español, fechas en formato dd-mm-aaaa y estados explícitos (*Pendiente de completar por el equipo*, parciales e implementados).

El control de versiones es el respaldo central del avance: la bitácora `REGISTRO_CAMBIOS_DOCUMENTACION.md` registra fecha, origen, acciones, verificación y archivos de cada ciclo documental (secciones 1–28); los commits del repositorio oficial del scraper muestran la línea de tiempo del backend (antecedente histórico: corrección del reloj de pruebas, commit `d201b0a`). La entrega debe excluir auxiliares LaTeX, dependencias y secretos, pero incluir los PDFs y fuentes evaluables. Esta revisión no certifica qué archivos están guardados en Git; el equipo debe registrar el commit y comprobar su contenido antes de entregar.

11.3. Roles planificados y trazabilidad individual pendiente

12. Trazabilidad documental

Cada afirmación de este documento proviene de las siguientes fuentes verificables:

Los cambios y pendientes vigentes de Fase 3 se resumen en el anexo `Novedades_desde_Fase2.md` de `Fase 3/Evidencias Grupales/` y en la bitácora `REGISTRO_CAMBIOS_DOCUMENTACION.md` de la raíz.

Referencias

Este documento cita los siguientes estándares y fuentes primarias (formato, estándar/autor, nombre, versión o RFC y URL de consulta):

[1] World Wide Web Consortium. *Web Content Accessibility Guidelines (WCAG) 2.2*. W3C Recommendation, oct. 2023. https://www.w3.org/TR/WCAG22/.

[2] Hardt, D. (ed.). *The OAuth 2.0 Authorization Framework*. RFC 6749, IETF, 2012. https://www.rfc-editor.org/rfc/rfc6749.

[3] Sakimura, N., Bradley, J. y Jones, M. *Proof Key for Code Exchange* (PKCE). RFC 7636, IETF, 2015. https://www.rfc-editor.org/rfc/rfc7636.

[4] PostgreSQL Global Development Group. *PostgreSQL Documentation: Row Security Policies*. https://www.postgresql.org/docs/current/ddl-rowsecurity.html.

[5] OpenStreetMap Foundation. *Nominatim Usage Policy* (geocodificación de respaldo del mapa). https://operations.osmfoundation.org/policies/nominatim/.

[6] Proyecto Mermaid. *Mermaid: documentación oficial* (herramienta utilizada para los diagramas propios 01–07). https://mermaid.js.org/intro/.

[7] Google LLC (equipo Angular). *Angular Style Guide* y documentación de componentes standalone (v20–22). https://angular.dev/style-guide.

[8] Ionic. *Ionic Framework documentation* (v9) para navegación responsiva móvil/escritorio. https://ionicframework.com/docs.

[9] Supabase. *Supabase documentation* (Row Level Security, Edge Functions y Inicio de sesión con GoTrue). https://supabase.com/docs.

[10] NVIDIA Corporation. *NVIDIA NIM for microservices* (API de inferencia de OCR y redacción). https://www.nvidia.com/en-us/ai-data-science/products/nim-microservices/.

Estándar / práctica	**Dónde se aplica**	**Evidencia verificable**
Patrones orientados a WCAG 2.2 en las interfaces (avisos semánticos, foco visible, objetivos táctiles, movimiento reducido)	Login, recuperación de contraseña y publicación de eventos	`role=.ªlert"` y `role="status"` con botón de cierre y tokens de color; Novedades de Fase 3, sección 6.
OAuth 2.0 y PKCE (RFC 6749, RFC 7636) en el regreso de proveedores externos	`/auth/callback` (Angular Router)	`auth-callback.page.ts` y flujo `signInWithOAuth` de supabase-js.
Control de acceso en la base de datos por mínimo privilegio (RLS de PostgreSQL)	Modelo documentado de catálogo y perfil; validar políticas remotas	Migraciones SQL (repositorio oficial del scraper) y vista `public_event_catalog`.
Sincronización idempotente por upsert y conciliación configurable	`SupabaseExporter` del scraper	Pipeline por lotes configurable y pruebas `test_supabase.py`.
Integración continua con compuerta de calidad (las pruebas bloquean la publicación de datos)	Workflow `scrape-events.yml`	Run #12: falla de pruebas impide extracción/sincronización; runs #25 en verde.
Separación de secretos por entorno (configuración 12-factor, sin claves en el cliente)	Backend y despliegues	`.env.example` en todos los entregables, GitHub Secrets y Edge Functions `admin-account` y funciones de recuperación con clave `service_role` solo en Supabase.
Patrones de arquitectura de software (Strategy, Factory, Repository, Adapter, Chain of Responsibility, Bulkhead, Cache-Aside)	Scraper y frontend Angular	Documentación interna de ambos entregables y pruebas con inyección de dependencias; servicios Angular con inyección estándar.
Convenciones de estilo y verificación automática de dependencias	Repositorios del proyecto	Python snake_case con `requirements.txt`; Angular 22 standalone con `angular-eslint` y `package-lock.json`; SCSS por token de diseño (`_tokens.scss`).
Responsividad y app instalable	Frontend web y Android	Navegación dual móvil/escritorio de Ionic, `manifest.webmanifest` y el build APK de Capacitor (Novedades de Fase 3, sección 4).

Cuadro 4: Estándares de desarrollo industriales adoptados con su evidencia en el repositorio.

Integrante	Área de responsabilidad según planificación
Roberto Riffo (líder)	Frontend Angular/Ionic (páginas, layouts y design system), gestión del proyecto y documento de metodología.
José Miguel Moncada	Base de datos, plan de pruebas y ejecución QA (ciclo del 11-10-2026 con 13 escenarios y matriz de defectos); recomendaciones e informe QA de Fase 3.
David Herrera	Backend e integraciones: scraper, su CI `scrape-events.yml`, migraciones SQL y Edge Functions `admin-account` / `reset-password`.

Cuadro 5: Distribución planificada según el design brief; no certifica autoría de cambios. Adjuntar commits o actas reales para demostrar aportes.

Sección	Evidencia utilizada
Arquitectura general, scraper y frontend	Entregables del scraper y frontend incluidos como evidencias de sistema en Fase 2; `package.json`, `app.routes.ts`, `supabase.service.ts`, `api.service.ts`, Edge Functions y diagramas técnicos de Fase 2. Integración frontend–Supabase implementada y verificada en TypeScript (catálogo real directo, autenticación y OAuth preparado).
Modelo de datos	Documentación de base de datos incluida en el entregable del scraper de Fase 2, modelo DBML, diccionario de tablas y `DB_evento_cultural.pdf` (Fase 1).
Diagramas UML	Rutas y servicios reales del frontend; clases y patrones documentados del scraper; matriz MoSCoW de Fase 1. Los siete diagramas (01–07) comparten estándar visual, fueron pulidos y reexportados el 27-09-2026 y el ER (06) fue actualizado el 03-10-2026.
Requisitos no funcionales	Matriz MoSCoW de requerimientos (Fase 1) y evidencia de código por requisito.
Pruebas	Carpeta `tests/` del scraper (14 módulos, aproximadamente 1.900 líneas), sección “Pruebas” de su README y configuración Vitest del frontend.
Docker	Inspección completa de ambos entregables (sin Dockerfile ni compose) y `CONTENIDO_DEL_REPOSITORIO.md`.
Innovación	Design brief de Fase 1, matriz MoSCoW, tabla de fuentes del README del scraper y funcionalidad “para ti” del frontend.

Cuadro 6: Trazabilidad entre secciones del documento y evidencia del repositorio.